Khai thác tập phổ biến bằng Eclat: Cài đặt Julia, đánh giá và ứng dụng Groceries – Splice DNA

Nguyễn Đức Hưng Trần Thiên Phúc
Môn Khai phá dữ liệu và ứng dụng
Khoa Công nghệ Thông tin, Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM
Bản review phần A; Chương 3 chưa hoàn tất

***Tóm tắt nội dung*—Báo cáo trình bày ECLAT bằng Julia với hai biểu diễn tidsets: sorted lists và BitVector. Cả hai dùng cùng DFS prefix extension và trả mọi frequent itemset cùng support tuyệt đối. Phần cài đặt phân biệt rõ lịch DFS của nhóm với bottom-up trong paper Zaki (2000), đồng thời cung cấp parser/writer SPMF và threshold chính xác. Kiểm chứng kết hợp oracle vét cạn độc lập, dữ liệu ngẫu nhiên và output thực của SPMF. Thực nghiệm trên Chess, Mushroom, Retail và T10I4D100K dùng sáu ngưỡng mỗi CSDL, năm repetitions; so sánh end-to-end riêng với read+mine trong Julia đã warm. Peak RSS được đo bằng hệ điều hành, cùng thí nghiệm Retail nested subsets và synthetic transaction widths. Các bảng và hình sinh từ CSV/logs thực, không giới hạn output để giảm thời gian. Phân tích tập trung vào mật độ tidsets, chi phí intersection, allocation và ảnh hưởng runtime nền; không giả định BitVector luôn có lợi trên dữ liệu thưa. Đây là bản review Chương 1–2 của thành viên A; Chương 3 về Groceries và Splice DNA thuộc thành viên B còn ở trạng thái template, nên PDF chưa phải báo cáo nghiệm thu cuối của nhóm.**

***Index Terms*—frequent itemset mining, Eclat, Julia, BitVector, association rules, splice DNA**

I. Chương 1: Cài đặt thuật toán ECLAT

A. Bài toán và phạm vi cài đặt

Khai thác tập phổ biến (frequent itemset mining, FIM) tìm các tổ hợp item xuất hiện trong đủ số quan sát. Kết quả là đầu vào của bài toán luật kết hợp: cùng một tập phổ biến có thể sinh nhiều luật có confidence và lift khác nhau [1]. Trong đồ án, thuật toán được viết bằng Julia, không gọi thư viện FIM để thực hiện mining. SPMF chỉ đóng vai trò tham chiếu độc lập và đối tượng so sánh thời gian. Hai backend của nhóm cùng trả mọi tập phổ biến và support tuyệt đối, gồm cả các tập không đóng và không tối đại. Việc giữ đầy đủ kết quả cần thiết để tra support của các tập con khi sinh luật, nhưng cũng làm chi phí bộ nhớ phụ thuộc output.

Gọi $D = (t_1, \ldots, t_n)$ là dãy n giao dịch; I là tập m item phân biệt; $t_i \subseteq I$ và $X \subseteq I$, $X \neq \emptyset$. Dùng dãy thay vì loại trùng các giao dịch: hai giỏ hàng giống nội dung vẫn là hai quan sát độc lập. Tidset và support count của X được định nghĩa bởi

$$T(X) = \{i \in \{1, \ldots, n\} : X \subseteq t_i\}, \quad \sigma(X) = |T(X)|. \quad (1)$$

Khi $n > 0$, relative support là $s(X) = \sigma(X)/n$. Với minsup tương đối $\theta \in (0, 1]$, cài đặt dùng

$$\tau = \max\{1, \lceil \theta n \rceil\}, \qquad F = \{X \subseteq I : X \neq \emptyset,\ \sigma(X) \geq \tau\}. \quad (2)$$

Dấu bằng được giữ: support đúng ngưỡng vẫn frequent. CSDL rỗng hoặc $\tau > n$ trả vector rỗng; không định nghĩa relative support bằng phép chia cho không. API trong RAM chấp nhận giao dịch rỗng, vẫn tính nó trong n, còn blank line trong file được xem là khoảng trắng và không tạo giao dịch. Chính sách khác nhau này là tường minh, tránh thay đổi denominator khi đọc dữ liệu ứng dụng.

Ví dụ $\theta = 0.07, n = 100$ phải cho $\tau = 7$. CLI phân tích chuỗi thập phân thành `Rational{BigInt}`, rồi tính ceiling chính xác; không đi qua Float64. Dạng `40%` và `0.4` cùng biểu diễn $2/5$. Không nhận scientific notation hay giá trị ngoài $(0, 1]$ qua CLI. API mining nhận absolute count nguyên dương, tách biệt quyết định ngưỡng khỏi thuật toán.

B. Biểu diễn vertical và phân hoạch prefix

Trong horizontal layout, mỗi hàng chứa các item của một transaction. Vertical layout lưu danh sách TID cho từng item; TID là chỉ số 1-based sau bước chuẩn bị. Công thức then chốt là

$$T(X \cup Y) = T(X) \cap T(Y). \quad (3)$$

Một TID nằm ở vế trái khi transaction chứa cả X lẫn Y, đúng với điều kiện nằm trong cả hai tập ở vế phải. Bởi vậy không cần quét toàn bộ horizontal database để đếm từng candidate. Nếu $X \subseteq Y$ thì $T(Y) \subseteq T(X)$, suy ra $\sigma(Y) \leq \sigma(X)$. Đây là tính anti-monotone dùng để prune: candidate có support nhỏ hơn τ không có frequent descendant.

Cố định item order theo ID tăng dần. Với prefix tăng P, các atom trong một lớp có dạng $P \,\|\, x$, trong đó $\|$ là phép nối tuple. Mỗi atom lưu key đầy đủ và tidset chính xác của key; extension x tăng theo thứ tự của lớp. Chỉ ghép atom hiện tại với sibling đứng sau. Lớp con cho $P \,\|\, x$ chứa các $P \,\|\, x \,\|\, y$ đạt ngưỡng với $y > x$. Phân hoạch này vừa cung cấp đường đi canonical vừa giới hạn số lớp trung gian phải xử lý đồng thời.

Thuật toán 1 DFS prefix extension chung cho hai backend

Require: Các atom S có cùng prefix, tăng theo extension; ngưỡng τ

function DFS(S, τ, F)
  for $i = 1, \ldots, |S|$ **do**
    $a \leftarrow S_i$
    thêm $(a.key, \text{support}(a.tids))$ vào F
    $C \leftarrow [\,]$
    for $j = i + 1, \ldots, |S|$ **do**
      $b \leftarrow S_j$
      $U \leftarrow \text{intersect}(a.tids, b.tids)$
      if $\text{support}(U) \geq \tau$ **then**
        $K \leftarrow a.key \,\|\, \text{last}(b.key)$
        thêm atom (K, U) vào C
      end if
    end for
    if C khác rỗng **then**
      DFS(C, τ, F)
    end if
  end for
end function

C. Liên hệ với Zaki (2000) và lựa chọn adaptation

Nguồn thuật toán chính là Zaki [2]: mục 4.1 trình bày support counting qua tid-lists; mục 4.2 phân rã lattice theo prefix; mục 5.4 mô tả Eclat bằng prefix equivalence và bottom-up search. Fig. 12 dựng các lớp con trước khi gọi đệ quy. Nhóm giữ nguyên cơ sở giao tidsets và phân hoạch, đồng thời chọn duyệt theo chiều sâu (depth-first search, DFS): xử lý hết một lớp con trước sibling. Thuật toán 1 phản ánh mã nhóm và không phải bản chép nguyên Fig. 12. Tài liệu SPMF cũng mô tả một phiên bản Eclat theo DFS [3].

Một lựa chọn khác liên quan F_2. Trong mã nhóm, các cặp singleton frequent được giao trực tiếp trong RAM. Nhóm không triển khai bước vertical-to-horizontal recovery và bộ đếm cặp tam giác của mục 5.1. Vì vậy kết quả giữ cùng ngữ nghĩa FIM nhưng không được xem là tái tạo toàn bộ các tối ưu và kiến trúc lưu trữ của paper. Không dùng diffsets, maximal-clique partitioning hay chỉ xuất maximal sets. Ghi chú chi tiết và ánh xạ từng mục paper nằm trong `docs/paper-notes.md`.

D. ECLAT Basic: verticalization và phép giao

Backend Basic ở `src/algorithm/baseline.jl`. Duyệt từng transaction theo `enumerate`, thêm TID vào dictionary từ item đến `Vector{Int}`. Validation đã bảo đảm mỗi item có tối đa một lần trong một transaction; TID được thêm tăng dần nên lists tự sorted, không phải sort lại. Các singleton dưới τ bị loại khỏi danh sách atom; các key còn lại được sort tăng ID. Một transaction lặp nội dung vẫn được thêm với TID mới, đúng định nghĩa tại (1).

Phép `intersect_tids` dùng hai con trỏ i, j, bắt đầu tại vị trí đầu của hai lists A, B. Nếu $A_i = B_j$, ghi TID và tăng cả hai; nếu nhỏ hơn, chỉ tăng con trỏ của giá trị nhỏ hơn. Dừng khi một list hết. Các phần tử đã đi qua không còn khả năng khớp với phần suffix của list kia, nên quy tắc không bỏ sót giao. Mỗi con trỏ tăng tối đa độ dài list; không dùng vòng lặp membership tuyến tính lồng nhau. Hai input chỉ đọc; kết quả cấp phát mới, có size hint bằng độ dài list ngắn. Cách này xử lý đúng cả giao rỗng lẫn một trong hai lists rỗng.

Bảng I
VÍ DỤ TỰ XÂY DỰNG: HORIZONTAL VÀ VERTICAL LAYOUT, $n = 4, \tau = 2$.

TID	Transaction	Item	TID-list
1	$\{1, 2, 3\}$	1	$[1, 2, 4]$
2	$\{1, 3\}$	2	$[1, 3, 4]$
3	$\{2, 3\}$	3	$[1, 2, 3]$
4	$\{1, 2\}$		

Bảng II
CÁC KIỂU DỮ LIỆU THỰC TẾ TRONG IMPLEMENTATION.

Kiểu	Nội dung chính
`Itemset`	Tuple ID dương tăng, không trùng
`Pattern`	Canonical key và support tuyệt đối
`ListAtom`	Full key và sorted `Vector{Int}`
`BitsetAtom`	Full key và `BitVector`
`PreparedDataset`	Transactions, mapping và metadata

Theo Bảng I, ba singletons có support 3. Nhánh đầu emit $\{1\}$; giao với item 2 cho $[1, 4]$ và với item 3 cho $[1, 2]$. Lớp prefix 1 gồm hai atom frequent $\{1, 2\}$ và $\{1, 3\}$. Sau khi emit $\{1, 2\} : 2$, phép giao hai atom con chỉ cho $[1]$: candidate $\{1, 2, 3\}$ có support 1 và bị prune. Quay lui, emit $\{1, 3\} : 2$. Nhánh item 2 còn sinh $\{2, 3\} : [1, 3]$, cuối cùng emit item 3. Kết quả có 6 itemsets. Ví dụ minh họa này khác fixture `basic.txt` thực tế, có 5 transactions và 7 patterns tại 40%; tách biệt hai ví dụ tránh gán nhầm expected.

E. Kiểu dữ liệu, API và xử lý vào/ra

Bảng II tóm tắt các kiểu của schema v1.0.0. Hai backends dispatch từ `mine` sang representation cụ thể, nhưng gọi chung `enumerate_class!`. Vì vậy ordering, phát candidate và pruning không bị thay đổi giữa hai phép đo. Các vector atom và tidset trong hot loop có kiểu cụ thể; key có độ dài thay đổi theo itemset theo hợp đồng dùng chung.

API gọi `mine(transactions,min_count)` với keyword `backend=:list`; backend còn lại là `:bitset`; trả `Vector{Pattern}`. Parser đọc numeric SPMF TXT, bỏ blank và các dòng bắt đầu bằng `#`, `%`, `@`. Item không dương, item lặp hoặc dòng không tăng bị reject; parser không âm thầm chuẩn hóa lỗi. Input caller không bị thay đổi. Các bước chuẩn bị benchmark chuyển ID 0 về ID dương được ghi nhận riêng ở Chương II.

Writer sắp theo độ dài, sau đó tuple items rồi ghi `1 2 #SUP: 3`. Output ổn định độc lập thứ tự DFS và bị từ chối nếu có key lặp. Comparator đọc về dictionary key canonical và support nguyên; reject duplicate output kể cả khi dòng có cùng support. Với fixture `basic.txt`, hai lựa chọn CLI `-minsup 0.4` và `-min-count 2` cho cùng byte output. Các lệnh chạy trong `scripts/mine.jl`; CLI bắt lỗi option trùng, thiếu

tham số, hai dạng support cùng lúc và output trùng input. Test round trip chạy cả hai backend.

F. Tính đúng đắn và quản lý bộ nhớ

Invariant về support. Singleton tidsets đúng theo cách dựng từ từng TID. Giả sử các atom ở một lớp chứa tidset đúng. Khi ghép hai atom cùng prefix, (3) cho tidset chính xác của key union. Chỉ candidate đạt τ được đưa vào lớp con, nên mọi Pattern được emit có support đúng và frequent.

Không trùng. Một key canonical $x_1 < \cdots < x_k$ chỉ có một chuỗi prefix $x_1, x_1x_2, \ldots, x_1 \cdots x_k$. Quy tắc chỉ ghép later siblings không cho đường hoán vị hoặc sibling khác tạo lại cùng key. Không emit empty prefix.

Đầy đủ. Nếu X frequent thì mọi tập con cần cho các bước giao tạo prefix của X đều frequent. Chúng không thể bị prune. Theo quy nạp trên độ dài prefix, DFS sẽ xét đúng extension tiếp theo và cuối cùng emit X. Như vậy thuật toán xuất mọi frequent itemset, không chỉ các phần tử cực đại của lattice.

Mỗi candidate có tidset mới; không chia sẻ buffer mutable với parent. Khi đệ quy trở về, lớp con hết được sử dụng có thể được garbage collector thu hồi. Tuy nhiên các ancestor classes vẫn đang hoạt động; không áp dụng bound "chỉ hai mức" của một lịch duyệt khác. Output vector giữ toàn bộ Patterns đến khi mining kết thúc; writer cũng tạo vector được sắp thứ tự. Các phần này cùng tham gia peak RAM.

G. Tối ưu BitVector

Backend ở `src/algorithm/bitset.jl` đếm singletons trước, chỉ cấp bitmap cho items đạt τ, sau đó duyệt transactions lần nữa để đặt bit. Với n giao dịch, bit thứ i bằng 1 khi item xuất hiện ở TID i. BitVector là mảng bit đóng gói của Julia [4]; phép `a .& b` tạo giao mới và `count` đếm bit được đặt. Cài đặt sử dụng API công khai, không thao tác trực tiếp internal chunks. Các độ dài không chia hết cho word size được kiểm tra riêng.

Ở bước đệ quy, chỉ phép giao và hàm support khác Basic. Không dùng in-place AND vào mask cha, không giới hạn độ dài pattern và không thay đổi item ordering để tạo lợi thế benchmark. AND rồi count hiện là hai thao tác; chưa fuse chúng hay reuse candidate buffers. Hai cách biểu diễn do đó có cùng cây liệt kê và cùng kết quả, nhưng chi phí xử lý tidsets khác nhau. Correctness về ownership được kiểm bằng repeated calls, invariance khi đảo transactions và oracle trên dữ liệu ngẫu nhiên.

H. Phân tích thời gian và bộ nhớ

Gọi $L = \sum_{i=1}^{n} |t_i|$, $m_1 = |F_1|$, C là tập candidate intersections thực sự được thực hiện, và w là số bit/word. Với $c \in C$, hai tidsets đầu vào là $T_{c,a}, T_{c,b}$. Basic có chi phí giao

$$T_{\text{join,list}} = O\left(\sum_{c \in C} (|T_{c,a}| + |T_{c,b}|)\right). \tag{4}$$

Verticalization cần $O(L)$ và sort IDs cần $O(m \log m)$. BitVector dựng masks với chi phí $O(L + m_1\lceil n/w \rceil)$ ngoài sorting, còn tổng giao/count là

$$T_{\text{join,bit}} = O(|C|\lceil n/w \rceil). \tag{5}$$

Hai công thức không bao gồm tất cả chi phí: validation và hashing input; tạo key; constructor Pattern sort/validate key; GC; writer sort và serialization. Riêng normalization các key có tổng bound $O(\sum_{X \in F} |X| \log |X|)$ và ghi output cần ít nhất số item IDs $\sum_{X \in F} |X|$. Trường hợp mọi tập con frequent có $|F| = 2^m - 1$; không có bound tuyến tính vô điều kiện. Các kết quả graph-theoretic với giả thiết sparsity ở Appendix A của paper không thay thế phân tích cài đặt này.

Bộ nhớ gồm horizontal input $O(L)$, vertical data, active classes, một giao tạm và output. Một list dùng khoảng $8|T|$ byte payload trên máy 64-bit, còn một bitmap cần $8\lceil n/64 \rceil$ byte cộng overhead object/array. So payload, điểm cân bằng xấp xỉ $|T|/n = 1/64$; đây chỉ là heuristic vì dictionary, allocation và cache còn ảnh hưởng. Lists co ngắn khi đi sâu vào sparse intersections, bitmap vẫn dài n. BitVector có thể giảm chi phí trên dense cases nhưng không được giả định luôn nhanh hơn hoặc luôn giảm resident set size (RSS). Peak process RSS còn gồm runtime và biên dịch just-in-time (JIT); dữ liệu cấp phát tích lũy từ `@timed` là metric khác.

I. Kiểm thử, association rules và tái lập

Bộ kiểm thử mặc định bật cả hai backend và SPMF references. Oracle độc lập liệt kê mọi subset của tối đa 12 items rồi đếm trực tiếp trên horizontal transactions; không dùng phép giao của miner. Năm fixtures basic/dense/sparse/duplicates/boundary được kiểm ở count 1, count 2, n và $n + 1$. Thêm 20 DB ngẫu nhiên seed 42, CSDL/transaction rỗng, một item, ID không liên tiếp, database dày, input invalid, caller ownership và các kích thước 63/64/65/127/128/129 giao dịch. Các test cuối cho cả phần mining, schema, rules và CLI được lưu nguyên stdout.

Reference tại 40% được tạo thực sự bằng SPMF 2.64b, pin JAR SHA-256, input hash, output hash và lệnh trong manifest. Comparator xác minh toàn bộ key/support, đếm missing, extra, wrong support; hai output rỗng vẫn là một trường hợp hợp lệ, không tự suy correctness của các trường hợp có output từ ví dụ rỗng. Trên benchmark, runner còn đối chiếu từng cấu hình trước khi đo và kiểm output của từng repetition.

Helper `association_rules` dùng support dictionary đầy đủ, xét mọi phân hoạch $X = A \cup B$ với hai phía khác rỗng, rời nhau. Confidence được lọc bằng tỉ số counts chính xác trước khi xuất Float64; subset support thiếu là lỗi. Duyệt tập con đệ quy không phụ thuộc bitmask có số bit cố định. Luật được sort theo lift, confidence, support giảm dần, rồi length và keys tăng dần theo schema. Test dùng PreparedDataset để gọi mine và kiểm cả consequent nhiều item. Hai preprocessing pipelines Groceries/DNA thuộc Chương 3 vẫn cần thành viên B hoàn thiện; việc helper chạy đúng không được dùng để tuyên bố ứng dụng trên dữ liệu thực đã hoàn thành.

II. Chương 2: Thực nghiệm và đánh giá

A. Mục tiêu và câu hỏi thực nghiệm

Thực nghiệm trả lời năm câu hỏi: kết quả có khớp toàn bộ SPMF hay không; minsup làm thay đổi runtime và output như thế nào; BitVector có lợi trên dữ liệu nào; peak RAM thay đổi ra sao; và số giao dịch/độ rộng transaction tác động thế nào khi

Bảng III
GRID MINSUP CỐ ĐỊNH; NGUỒN: `configs/experiments.toml`.

CSDL	Minsup (%)
Chess	95, 90, 85, 80, 75, 70
Mushroom	60, 55, 50, 45, 40, 35
Retail	5, 4, 3, 2, 1, 0.5
T10I4D100K	2, 1.5, 1, 0.8, 0.6, 0.4

giữ minsup tương đối. Phân tích dùng dữ liệu được chạy bằng chính hai backends ở Chương I. Không lấy số runtime hoặc memory của paper làm số đo của nhóm. Các bảng LaTeX và hình được sinh trực tiếp từ CSV bởi `scripts/plot_results.py`; dữ liệu gốc, stderr/stdout và lệnh cho từng process được giữ để kiểm tra lại kết luận.

B. Phần cứng, phần mềm và cấu hình

Máy đo là Apple M1, 8 logical CPUs, RAM 8 GiB, macOS 14.6.1 arm64. Julia 1.10.10 chạy với một Julia thread, không dùng distributed execution; Java 23.0.1 chạy SPMF 2.64b. Controller dùng Python 3.14.0 và hình dùng Matplotlib 3.11.2. JAR tham chiếu được tải từ archive chính thức và pin SHA-256; Java 21 trở lên đọc được class version của JAR này. Bản JAR latest tại thời điểm thiết lập không chạy trên Java 23 nên không được dùng. Phiên bản và hash đầy đủ nằm trong `results/raw/environment.json` và manifest reference, không chỉ dựa vào tên file `spmf.jar`.

Bản mã thuật toán/harness cho lần đo được lưu bằng commit và source hashes trong `results/raw/protocol.json`. Seed là 42. Mỗi cấu hình có một preflight và năm repetitions; timeout mỗi process 120 giây. SPMF dùng `-Xmx2g`. Grid gồm 24 cấu hình benchmark, 5 subsets và 6 widths; output không giới hạn độ dài hay số lượng. Các trường hợp không hoàn tất có status riêng, không dùng runtime/pattern count bằng 0 để biểu diễn timeout. Các ngưỡng chính ở Bảng III được pilot trước trên bốn CSDL; pilot không gộp vào dữ liệu đo chính thức.

C. Nguồn dữ liệu, chuẩn hóa và thống kê thực

Bốn CSDL được tải từ kho FIMI học thuật của Zaki [5]. Ưu tiên tìm snapshot HF/Kaggle nhưng không xác định được một bản chung phù hợp cho đủ bốn CSDL; sử dụng nguồn học thuật gốc, pin nội dung bằng raw SHA-256. Đây là snapshot theo hash, không phải tuyên bố một revision HF/Kaggle chưa xác minh. Acquisition script từ chối checksum khác để tránh chạy trên dữ liệu đã đổi. Dữ liệu raw và normalized không commit vào Git; repo giữ URL, hash, license notes và mã tải tái lập.

Server trả gzip ngay tại URL kết thúc bằng `.dat`; script nhận diện magic bytes trước giải nén. Chess/Mushroom vốn có ID dương. Retail và T10I4D100K có ID 0, nên cộng 1 cho *mọi* item ID của hai CSDL đó để giữ hợp đồng parser. Đây là song ánh nên không đổi containment, support hoặc số patterns. Không bỏ dòng, không deduplicate transactions, không sửa parser để nhận input invalid. Cả Julia và SPMF nhận cùng byte file normalized; input SHA-256 được ghi vào từng row đo.

Bảng IV cho thấy không thể coi bốn CSDL có cùng đặc tính. Chess và Mushroom có width cố định lần lượt 37 và 23, với mật độ cao. Retail có nhiều item hiếm, width trung bình khoảng 10.31 nhưng vocabulary 16,470 items. T10I4D100K có 100,000 giao dịch trong đúng snapshot này; không thay con số đó bằng thống kê của một phiên bản đã lọc ở nguồn khác. Mật độ được tính bằng

$$\rho = \frac{\sum_i |t_i|}{nm} = \frac{\bar{\ell}}{m}, \tag{6}$$

trong đó $\bar{\ell}$ là mean width. File manifest ghi điều kiện dùng dữ liệu Retail và attribution Tom Brijs và trích dẫn công trình được donor yêu cầu [6]; các nguồn còn lại không được gán giấy phép do nhóm tự suy đoán. Chỉ chia sẻ mã acquisition và thống kê, không tái phát hành raw.

D. Đối chiếu correctness trước timing

Với mỗi case, chạy ba backends trên cùng file, tính absolute threshold bằng (2), parse toàn bộ output về map canonical. Với hai maps A, R, định nghĩa số khớp $M = |\{X \in \mathrm{keys}(A) \cap \mathrm{keys}(R) : A[X] = R[X]\}|$ và

$$\mathrm{match_rate} = \frac{M}{|\,\mathrm{keys}(A) \cup \mathrm{keys}(R)|}. \tag{7}$$

Hai maps rỗng cho rate 1. Comparator còn đếm missing, extra và wrong support; exact equality yêu cầu cả ba bằng 0. Duplicate lines bị reject trước bước so sánh, không được dictionary ghi đè để che lỗi.

SPMF nhận double cho relative threshold. Để tránh sai số ceiling tại biên, controller gửi một giá trị ở giữa cùng absolute-count bin,

$$\theta_{\mathrm{SPMF}} = \frac{2\tau - 1}{2n}. \tag{8}$$

Với $n > 0$, $\lceil \theta_{\mathrm{SPMF}} n \rceil = \tau$, nên tiêu chuẩn frequent hoàn toàn giống ngưỡng đã khai báo; không giảm absolute minsup. Chuỗi tham số thật nằm trong command log. Toy references tại 40% dùng trực tiếp 40%; bộ test riêng xác minh count-bin conversion trên các kích thước benchmark và boundary values.

Bảng V thống kê các phép đối chiếu, không phải chỉ tỉ lệ bằng nhau về tổng số patterns. Sau preflight, mỗi output của các timed repetitions còn được so lại toàn bộ với map đã xác minh; worker warm cũng kiểm mỗi repetition. Thiếu/thừa/sai support làm runner dừng diễn giải hiệu năng của case đó. Với lỗi process hoặc timeout ở preflight, runner giữ bằng chứng lần thử thực và chuyển sang case khác, không tạo năm repetitions giả. Năm toy references, oracle vét cạn và randomized tests là lớp kiểm tra độc lập với grid thực nghiệm này.

E. Hai phạm vi thời gian và tính công bằng

End-to-end. Controller đo wall-clock từ ngay trước khi tạo process mới đến khi process kết thúc và controller nhận kết quả. Cả ba backends đều đọc input, mining đầy đủ và ghi output trong phạm vi đó; startup/JIT của Julia và startup JVM được tính. Một preflight làm ấm file cache và xác minh kết quả

Bảng IV
THỐNG KÊ TÍNH TỪ ĐÚNG FILE NORMALIZED: n TRANSACTIONS, m DISTINCT ITEMS, TB/MAX LÀ SỐ ITEM/GIAO DỊCH; NGUỒN: `RESULTS/RAW/DATASET_STATISTICS.CSV`.

CSDL	n	m	TB	Max	Mật độ (%)
Chess	3196	75	37.00	37	49.333
Mushroom	8124	119	23.00	23	19.328
Retail	88162	16470	10.31	76	0.063
T10I4D100K	100000	870	10.10	29	1.161

Bảng V
ĐỐI CHIẾU TOÀN BỘ MAP TRÊN GRID BỐN BENCHMARK, MỖI NGƯỠNG CÓ BASIC/SPMF VÀ BITVECTOR/SPMF; NGUỒN: `RESULTS/CORRECTNESS/COMPARISONS.CSV`.

CSDL	Đối chiếu	Khớp đủ	Sai lệch
Chess	12	12	0
Mushroom	12	12	0
Retail	12	12	0
T10I4D100K	12	12	0

nhưng không được tính là repetition. Timer bao gồm overhead nhỏ của OS monitor/controller; cơ chế wait có timeout của Python có thể thêm tối đa khoảng 50 ms ở cuối lần chạy. Do đó đây là chi phí thực thi CLI do harness quan sát, không phải timer chỉ của thuật toán. Không so internal timer trong log SPMF với thời gian Julia ở phạm vi khác.

Internal read+mine. Trong process Julia riêng cho mỗi case/backend, worker compile ví dụ nhỏ rồi warm-up toàn bộ case một lần. Sau đó năm lần đo đọc lại file, validation, verticalization và DFS bằng `@timed`; mỗi lần có `GC.gc()` trước timer. Ghi output, kiểm equality và GC chủ động ở ngoài phạm vi này. Bảng internal chỉ so hai backend Julia. Worker giữ map tham chiếu của warm-up để kiểm lặp; không gán RSS của cả batch cho từng lần internal. Chưa có Java warm harness tương đương nên không đưa SPMF vào đường cong internal.

Mỗi điểm đồ thị biểu diễn median năm lần; error bars là min–max, không diễn giải thành confidence interval. Hai phạm vi được vẽ ở hai panels với đơn vị rõ ràng. Tỉ số tối ưu được tính chỉ với cùng metric scope và các phép đo hợp lệ:

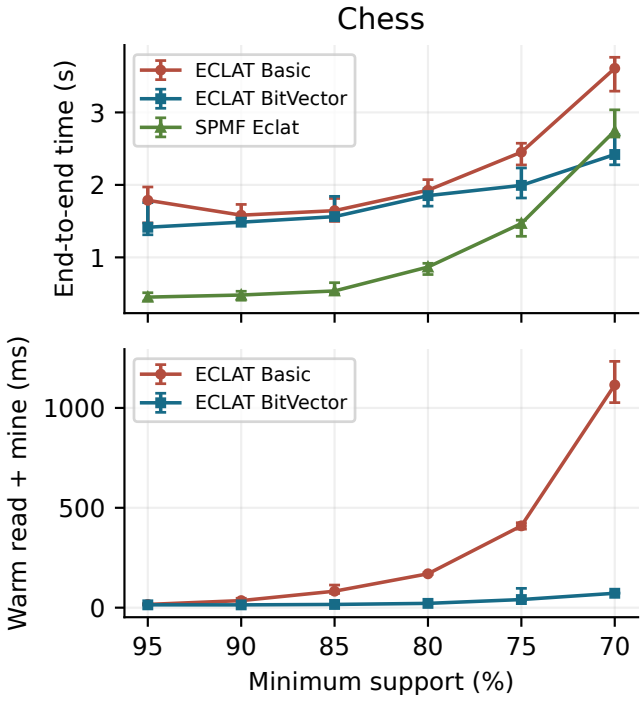
$$S = T_{\text{Basic}}/T_{\text{BitVector}}. \tag{9}$$

$S > 1$ có lợi cho BitVector; $S < 1$ được giữ nguyên, không bỏ khỏi báo cáo.

F. Runtime theo minimum support

Hình 1–4 trình bày đủ sáu ngưỡng mỗi CSDL. Panel trên dùng chung boundary end-to-end cho Basic, BitVector và SPMF. Panel dưới làm rõ chi phí đọc+mine trong runtime Julia đã warm. Khác biệt giữa hai panels cho biết startup/compilation/serialization có thể che một phần cải thiện của phép giao. Vì chúng đo những công việc khác nhau, không lấy hiệu hai median để khẳng định chính xác thời gian JIT hoặc thời gian ghi file.

Bảng VI và Bảng VII giúp đánh giá cùng ngưỡng dưới hai góc nhìn. Ở các ngưỡng trung bình, median warm

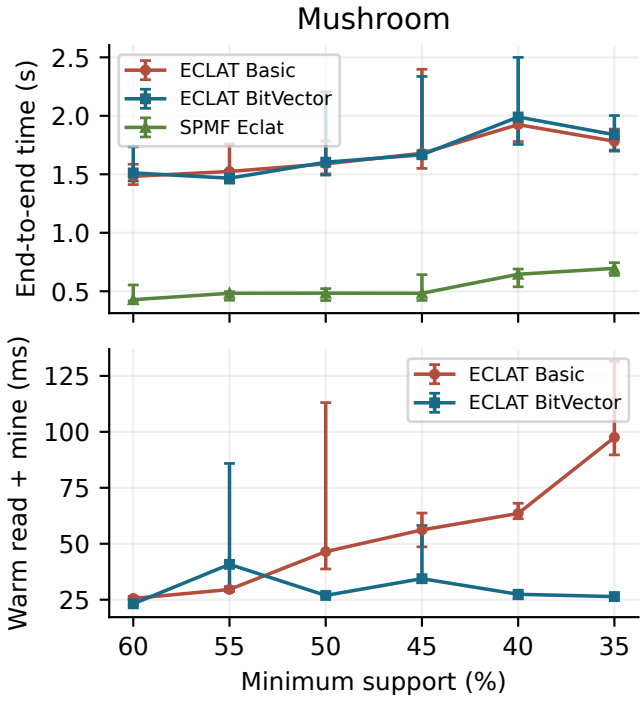


Hình 1. Chess: end-to-end (giây) và warm read+mine (ms), median và min–max của năm lần; nguồn: `results/raw/benchmark_runs.csv`.

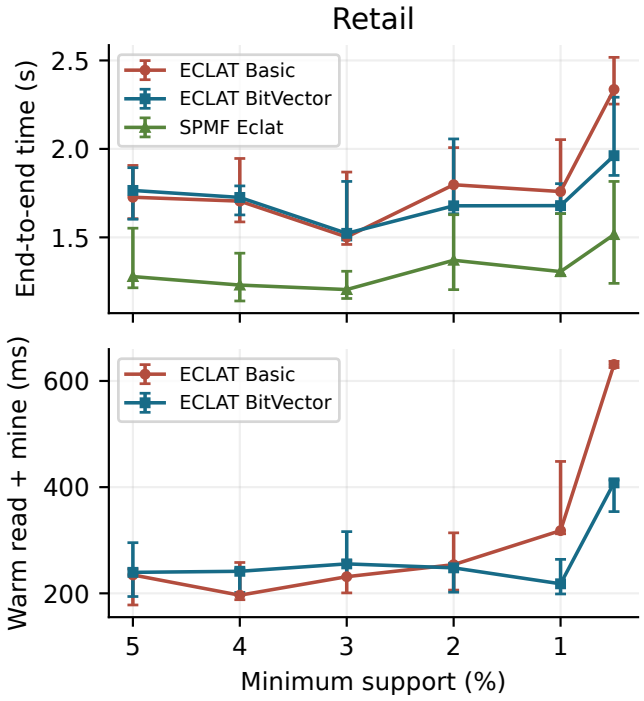
Bảng VI
WARM READ+MINE TẠI MINSUP TRUNG BÌNH: CHESS 85%, MUSHROOM 50%, RETAIL 3%, T10I4D100K 1%. S THEO (9); NGUỒN: `OPTIMIZATION.CSV`.

CSDL	τ	Basic (ms)	BitVector (ms)	S
Chess	2717	82.50	16.16	5.10
Mushroom	4062	46.48	26.92	1.73
Retail	2645	231.34	255.48	0.91
T10I4D100K	1000	1723.08	552.79	3.12

của Basic/BitVector lần lượt là 82.50/16.16 ms trên Chess, 46.48/26.92 ms trên Mushroom, 231.34/255.48 ms trên Retail và 1723.08/552.79 ms trên T10I4D100K. Tỉ số tương ứng xấp xỉ 5.11×, 1.73×, 0.91× và 3.12×; giá trị dưới 1 của Retail được giữ để phản ánh median BitVector chậm hơn tại 3%. Với Chess, tid-lists của nhiều items dài và phần search/output tăng khi hạ minsup, nên packed AND giảm nhiều công việc quét TID. Với Retail ở ngưỡng cao, số frequent patterns nhỏ, chi phí đọc/validation/verticalization chiếm phần lớn; count trước



Hình 2. Mushroom: cùng hai phạm vi đo của Hình 1, năm lần mỗi điểm, output đầy đủ. Nguồn: raw benchmark CSV.



Hình 3. Retail: runtime theo minsup; error bars min–max, không coi timer JVM nội bộ là một phần của panel warm Julia. Nguồn: raw benchmark CSV.

Bảng VII
MEDIAN END-TO-END TẠI CÙNG CÁC NGƯỠNG CỦA BẢNG VI; NGUỒN: `RESULTS/RAW/END_TO_END_MEDIUM.CSV`.

CSDL	Basic (s)	BitVector (s)	SPMF (s)
Chess	1.644	1.562	0.537
Mushroom	1.589	1.603	0.484
Retail	1.502	1.523	1.205
T10I4D100K	3.451	2.105	0.884

rồi dựng bitmap cũng thêm một lượt dữ liệu. Kết quả này cần được giải thích bằng (4)–(5), thay vì mặc định bitmap thắng trên mọi CSDL thưa. SPMF là cài đặt Java trưởng thành, có representation và overhead riêng; end-to-end tốt hơn không tự chứng minh một phép AND của Java nhanh hơn Julia.

G. Số frequent itemsets theo minsup

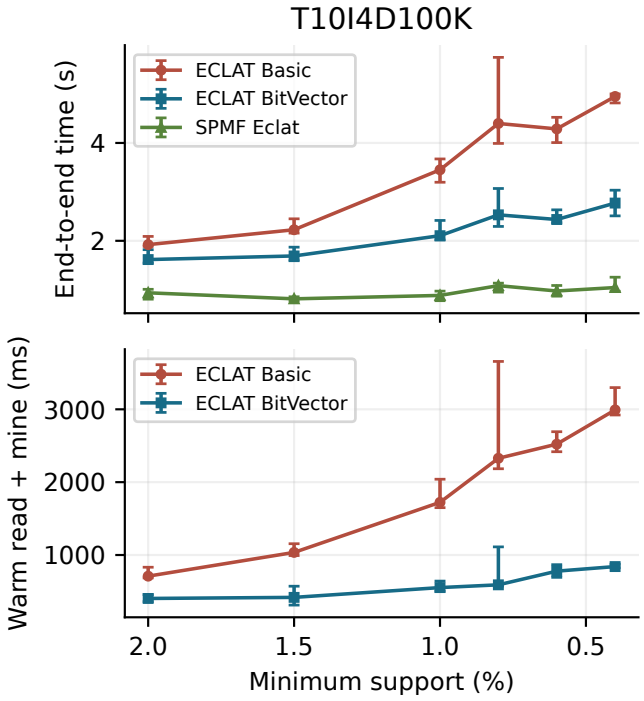
Hình 5 dùng đúng các outputs đã kiểm chứng, không truncate. Trên sáu ngưỡng từ cao xuống thấp, Chess tăng từ 77 lên 48,731 patterns; Mushroom từ 51 lên 1,189; Retail từ 16 lên 580; T10I4D100K từ 155 lên 2,001. Các CSDL dùng grids minsup khác nhau, nên không quy toàn bộ chênh lệch counts giữa CSDL cho riêng density. Khi hạ τ trên một CSDL cố định, một itemset đã frequent không thể trở thành infrequent, nên output size không giảm. Tuy nhiên số frequent singletons không đủ dự đoán số patterns: những kết hợp nhiều item trên dữ liệu dày có thể cùng đạt ngưỡng và phát triển theo tổ hợp. Ngược lại, nhiều item hiếm của Retail không thể được mở rộng ở các ngưỡng cao của grid. Minsup bằng nhau về tỉ lệ cũng không làm candidate space của các CSDL bằng nhau.

Việc giảm support vừa giữ thêm candidates vừa làm thay đổi độ dài tidsets có thể giữ trong các lớp. Với Basic, rất nhiều giao lists dài tại các prefix ngắn có thể chi phối tổng thời gian; tại prefix sâu, lists co lại và có thể trở nên hiệu quả. Với BitVector, mỗi giao vẫn có $\lceil n/w \rceil$ words, nên lợi thế phụ thuộc cả số joins chứ không chỉ số patterns cuối. Báo cáo chưa instrument riêng candidate count theo độ sâu; các giải thích trên là cơ chế phù hợp với mã và số đo, không phải kết quả profiling từng phase đã thực hiện.

H. Peak RAM và hiệu quả bộ nhớ

Peak RSS đo bằng `/usr/bin/time -l` của macOS trên từng process end-to-end, chuyển bytes sang MiB bằng 2^{20}. Monitor bao trọn startup, compilation, horizontal input, vertical data, intermediate tidsets, Patterns và writer. Đây là maximum resident set size của OS, không phải Julia allocation count hay SPMF heap log. Năm peak values được aggregate median tại minsup trung bình; raw CSV giữ mọi lần. Worker internal ghi RSS là NA vì không có peak riêng cho từng repetition.

Trong Bảng VIII và Hình 6, mức giảm được tính $100(1 - R_{\mathrm{BitVector}}/R_{\mathrm{Basic}})$ với R là median peak RSS. Do đó các mức giảm trên Chess, Mushroom và Retail lần lượt khoảng 14.88%, 4.63% và 3.89%. T10I4D100K có median 345.1 MiB của Basic so với 351.1 MiB của BitVector, tức mức giảm -1.74%. Chênh lệch nhỏ ở một phiên đo chưa chứng minh một regression bộ nhớ ổn định; cần xem spread raw và lặp trên các phiên khác. Do runtime nền và JIT tham gia peak, phần trăm RSS giảm không bằng phần trăm payload tidsets



Hình 4. T10I4D100K: runtime trên 100,000 transactions; panel trên giây, panel dưới ms; năm repetitions. Nguồn: raw benchmark CSV.

Bảng VIII
PEAK RSS TẠI CÙNG NGƯỠNG TRUNG BÌNH; GIẢM ÂM NGHĨA BITVECTOR DÙNG RSS LỚN HƠN. NGUỒN: RESULTS/RAW/MEMORY_COMPARISON.CSV.

CSDL	Basic (MiB)	BitVector (MiB)	Giảm (%)
Chess	304.4	259.1	14.9
Mushroom	271.6	259.0	4.6
Retail	291.4	280.0	3.9
T10I4D100K	345.1	351.1	-1.7

giảm. Với từ điển item lớn, bitmap có thể tiết kiệm nhờ chỉ cấp cho F_1, nhưng khi nhiều items đều frequent mà n lớn, mỗi bitmap vẫn phải phủ toàn bộ TIDs. Không ngoại suy các ngưỡng này thành cam kết tiết kiệm RAM ở minsup thấp tùy ý.

I. Scalability theo số transactions

Retail được hoán vị một lần bằng MersenneTwister seed 42; lấy prefixes của cùng hoán vị cho 10/25/50/75/100%. Vì vậy các samples nested, kể cả subset 100% được hoán vị so với file gốc. Minsup giữ ở 1%; τ được tính lại theo n thực của mỗi sample. Bảng IX giữ cả count output để không nhầm tăng input với tăng độ khó của search space.

Hình 7 gồm ba panels để tách output khỏi runtime. Đọc và verticalization phải xử lý nhiều item occurrences hơn khi n tăng; lists thường dài hơn và bitmaps cần nhiều words hơn. Nhưng sampling cũng thay support của các itemsets gần ngưỡng, trong khi ceiling có bước nhảy rời rạc. Vì vậy $|F|$ giữa các subsets có thể dao động dù input tăng: trong năm mẫu này

Bảng IX
RETAIL NESTED SUBSETS, MINSUP 1%, SEED 42; THỜI GIAN WARM MEDIAN. NGUỒN: RAW BENCHMARK CSV VÀ CASES.JSON.

Tỉ lệ (%)	n	$\|F\|$	Basic (ms)	BitVector (ms)
10	8816	161	27.20	21.83
25	22040	157	71.50	49.09
50	44081	162	141.53	109.54
75	66121	158	209.35	243.33
100	88162	159	301.48	226.03

Bảng X
SYNTHETIC UNIFORM: MỌI THAM SỐ NGOÀI WIDTH CỐ ĐỊNH; WARM MEDIAN, OUTPUT ĐẦY ĐỦ. NGUỒN: RAW BENCHMARK CSV VÀ GENERATOR SEED 42.

Width	τ	$\|F\|$	Basic (ms)	BitVector (ms)
5	2000	0	8.95	9.65
10	2000	0	17.08	16.32
15	2000	0	22.79	25.16
20	2000	53	71.90	31.63
25	2000	100	250.42	52.28
30	2000	100	309.60	51.10

counts nằm từ 157 đến 162, subset 100% có 159 patterns như Retail gốc tại 1%. Đường gần tuyến tính ở một grid cố định chỉ mô tả miền đo này; không chứng minh complexity của Eclat tuyến tính cho mọi D.

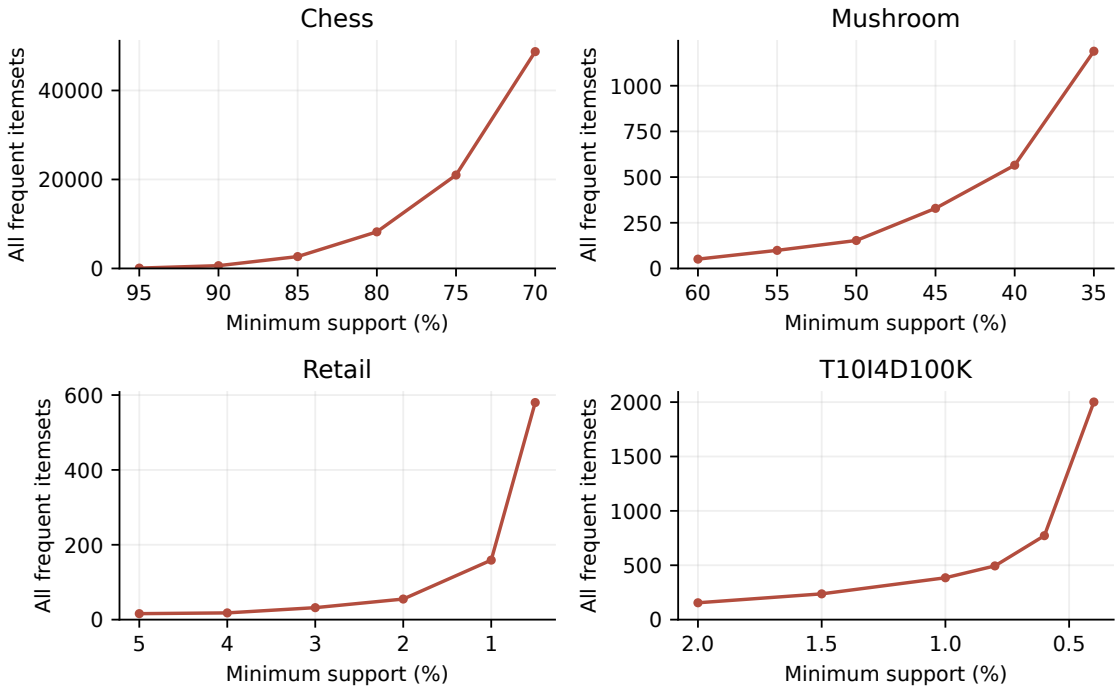
J. Ảnh hưởng của độ dài giao dịch

Generator của nhóm chọn đều không hoàn lại các items trong mỗi transaction bằng `randperm`, rồi sort; không phải IBM generator của paper. Thí nghiệm giữ $n = 10{,}000$, $m = 100$, seed 42 và $\theta = 0.20$, thay width $\ell \in \{5, 10, 15, 20, 25, 30\}$. Các CSDL khác width không được coi là nested. Không thêm planted patterns trong đợt chính; kết quả không có pattern vẫn được giữ vì nó kiểm tra rõ tác động của singleton filtering ở ngưỡng cố định.

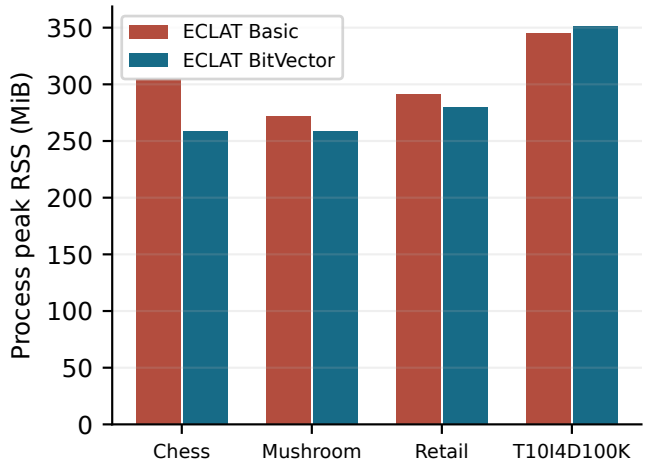
Bảng X và Hình 8 cần đọc cùng xác suất sinh dữ liệu. Theo đối xứng, expected support tương đối của một item là ℓ/m; của một cặp là $\ell(\ell-1)/(m(m-1))$. Khi width nhỏ hơn 20, expected singleton support dưới ngưỡng 20%; ở width 20 các singletons gần boundary nên một phần có thể vượt ngưỡng do sampling. Ngay width 30, expected pair support chỉ khoảng 8.79%, vẫn thấp hơn 20%. Do đó dữ liệu này chủ yếu kiểm tra đọc, verticalization, lọc F_1 và các pair intersections, không phải stress test các frequent patterns dài. Ít output không có nghĩa toàn bộ mining bằng 0 công việc: Basic vẫn dựng vertical lists cho mọi item, BitVector vẫn phải đếm singleton occurrences.

K. Giới hạn và các hướng tối ưu tiếp theo

Đợt đo chạy trên một máy cá nhân 8 GiB, không cố định CPU frequency/core affinity hay kiểm soát hoàn toàn nhiệt độ và tiến trình nền. Backend order cố định trong mỗi repetition có thể gây temporal bias; năm repetitions và min–max chỉ mô tả spread trong phiên này. Chưa đo trên máy khác hoặc fresh OS page cache. Các ngưỡng được chọn để toàn bộ output có



Hình 5. Số tất cả frequent itemsets theo minsup trên bốn CSDL. Counts lấy từ output Basic đã khớp toàn bộ BitVector/SPMF; nguồn: raw/summary CSV.



Hình 6. Median maximum process RSS (MiB), năm process độc lập mỗi backend/CSDL; ngưỡng như Bảng VI. Nguồn: raw memory comparison CSV.

thể kiểm chứng, không đại diện mọi miền minsup; chưa chạy đến vùng bùng nổ output không thể chứa trong RAM. Bất kỳ speedup nào chỉ áp dụng cùng dữ liệu, threshold và scope đã khai báo.

End-to-end bao gồm startup/JIT/JVM, serialization và overhead controller; warm Julia không so trực tiếp với SPMF vì thiếu Java harness cùng boundary. RSS là peak của toàn process, không tách riêng tidset payload và không phản ánh chỉ lượng allocation đang sống. Các synthetic cases độc lập đều không thay thế dữ liệu có cấu trúc đồng xuất hiện mạnh; muốn nghiên cứu deep frequent patterns cần thêm planted generator với protocol riêng. Correctness full-map ở các cases đã chạy không phải chứng minh thực nghiệm cho mọi CSDL có thể có; lập luận invariant ở Chương I bổ sung cho test.

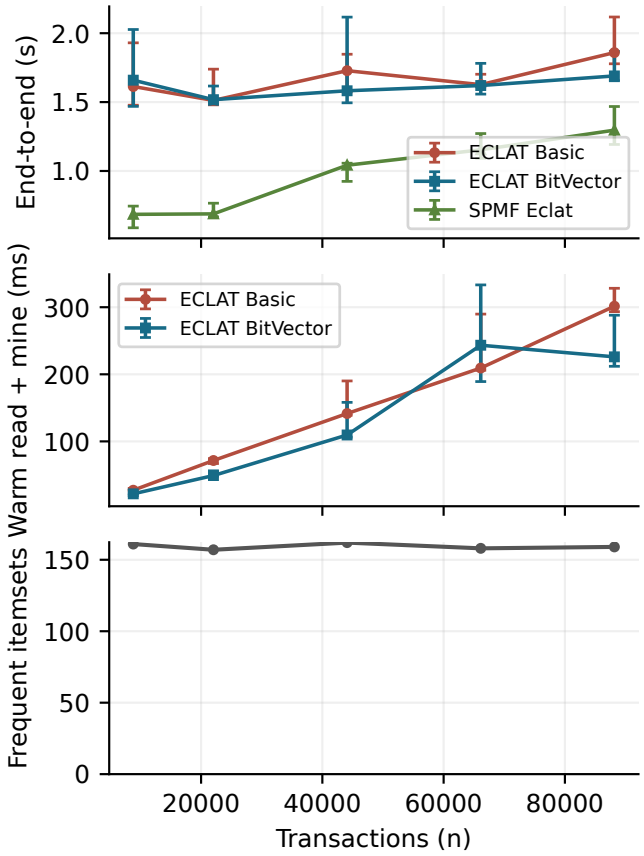
Hướng tối ưu thứ nhất là *adaptive representation*: chọn sorted list cho tidsets rất thưa và bitmap cho masks dày, theo payload/cost model được đo. Cần thêm mixed-intersection dispatch và kiểm lại ownership/correctness; điểm cân bằng $n/64$ về TID payload chỉ là giá trị khởi đầu. Thứ hai là *buffer reuse và fused AND/popcount*: giảm allocation của candidates bị prune, nhưng chỉ tái sử dụng buffer khi không còn parent/child tham chiếu. Có thể dùng buffer pool theo depth và đo GC/allocations cùng RSS để xác định lợi ích thực.

Hướng thứ ba là ordering theo support để thử giảm số giao ở các prefix khó, hoặc early-abort phép giao list khi số TID tối đa còn có thể khớp không đủ τ. Các thay đổi này chưa triển khai trong kết quả hiện tại; cần giữ canonical keys, cùng enumeration semantics và tạo protocol mới. Ở tầng đánh giá, randomize thứ tự backend, bổ sung Java warm harness và lặp qua nhiều phiên máy sẽ giúp kiểm tra độ ổn định của kết luận. Những hướng này là đề xuất từ cơ chế và số đo, không được tính là các tối ưu đã hoàn thành.

TÀI LIỆU

[1] R. Agrawal and R. Srikant, "Fast algorithms for mining association rules," in *Proceedings of the 20th International Conference on Very Large Data Bases*, 1994. [Online]. Available: https://www.vldb.org/conf/1994/P487.PDF

[2] M. J. Zaki, "Scalable algorithms for association mining," *IEEE Transactions on Knowledge and Data Engineering*, vol. 12, no. 3, pp. 372–390, 2000.



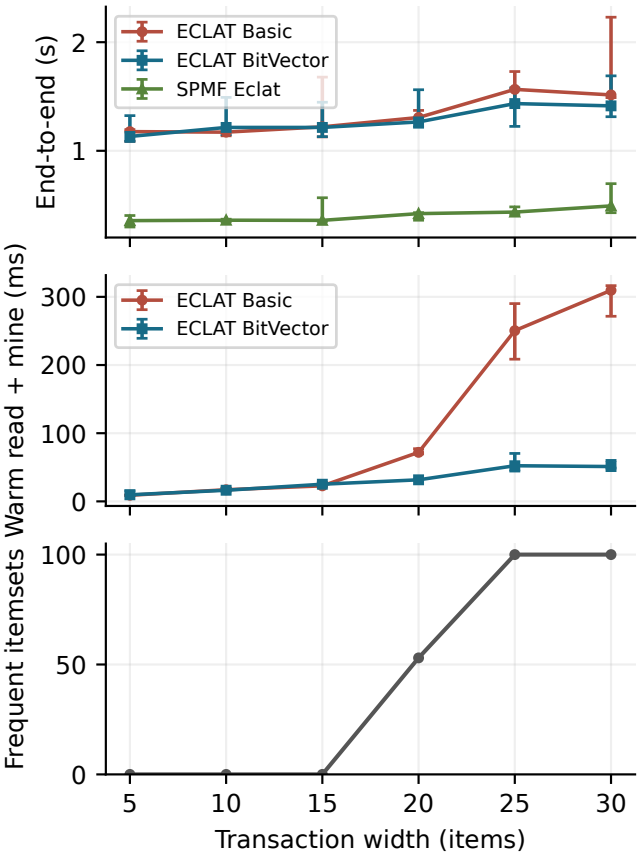
Hình 7. Retail: end-to-end (s), warm read+mine (ms) và output size theo n; relative minsup cố định 1%, năm repetitions; nguồn: raw benchmark CSV.

[3] P. Fournier-Viger, "SPMF: Mining frequent itemsets using the Eclat / dEclat algorithms," https://www.philippe-fournier-viger.com/spmf/Eclat_dEclat.php, truy cập ngày 30/10/2026.

[4] Julia Contributors, "The julia language: Arrays and BitArray," https://docs.julialang.org/en/v1/base/arrays/, aPI đối chiếu với Julia 1.10.10; truy cập 31/10/2026.

[5] "Frequent itemset mining dataset repository," https://www.cs.rpi.edu/~zaki/Workshops/FIMI/data/, nguồn học thuật; raw snapshots pin SHA-256 trong configs/datasets.toml; truy cập 31/10/2026.

[6] T. Brijs, G. Swinnen, K. Vanhoof, and G. Wets, "Using association rules for product assortment decisions: A case study," in *Proceedings of the Fifth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1999, pp. 254–260. [Online]. Available: https://sigmod.org/publications/discs/2000.1/out/p_usingassociatiotogik.htm



Hình 8. Synthetic width effects: end-to-end (s), warm read+mine (ms) và số patterns; $n = 10{,}000, m = 100, \theta = 0.20$, seed 42. Nguồn: raw benchmark CSV.